

WEEK3 Report

张筱涵

1. 实验思路

1.1 背景与动机

在 Week 2 中，BSM 模型的实证分析揭示了三个核心局限：波动率变异系数 $CV=0.62$ （远高于 BSM 假设的 0）、不同 VIX 制度间存在 40.1% 的价格差距、2020 年新冠危机期间定价误差高达 3 倍。这些发现说明 BSM 的静态 σ 假设在真实市场中存在系统性偏差，为机器学习模型的引入提供了明确的改进方向。Week 3 的核心目标是构建 ML 增强定价框架，通过动态捕捉市场特征来弥补 BSM 的不足，并通过严格的性能对比量化 ML 模型的实际提升效果。

1.2 两条技术路线

Approach 1: ML 波动率预测 + BSM 定价

该路线保留 BSM 的理论框架，仅用 ML 模型替换静态 σ 假设。具体做法是以市场多维特征（VIX 制度、利率动量、滚动相关性、情绪分数等）为输入，训练 Random Forest 预测动态波动率，再将预测的 σ 通过线性回归映射到 Chooser 期权价格。这条路线的优势在于保持了 BSM 的可解释性，同时赋予 σ 动态调整能力。

Approach 2: 端到端监督学习定价

该路线绕过 BSM 公式，直接以市场特征为输入、BSM 定价结果为目标变量，训练线性回归、随机森林（RF）、梯度提升树（GBDT）三个模型，学习从市场状态到期权价格的直接映射关系。这条路线的优势在于能够捕捉 BSM 公式无法建模的非线性特征交互。

1.3 情绪特征的引入

在 Week 1&2 的特征体系基础上，本周新增了市场情绪分数（Sentiment Score）作为额外特征。由于 NewsAPI 免费版仅支持近 30 天历史数据，无法覆盖 2018-2024 年区间，本项目采用 VIX 反向归一化作为市场情绪的代理指标：

$$\text{Sentiment Score} = 1 - \frac{\text{VIX} - \text{VIX}_{\min}}{\text{VIX}_{\max} - \text{VIX}_{\min}}$$

VIX 越高代表市场恐慌情绪越强（分数接近 0），VIX 越低代表市场情绪越平稳（分数接近 1）。

1.4 数据划分策略

为防止前视偏差，数据集按时间顺序严格划分为三部分：训练集（70%，2018-2022 年）、验证集（15%，2022-2023 年）、测试集（15%，2023-2024 年）。这种划分方式确保模型只能用历史数据预测未来，符合实际交易场景的信息约束。

1.5 模型评估框架

所有模型在相同测试集上通过三个指标进行评估：MAE（平均绝对误差）衡量定价偏差的平均水平；RMSE（均方根误差）对大误差更敏感，重点衡量极端情况下的定价准确性； R^2 衡量模型对价格变动的整体解释能力。

2. 关键代码和结果

2.1 情绪分数计算

```
df["Sentiment_Score"] = 1 - (df["VIX_Close"] - df["VIX_Close"].min()) / (
    df["VIX_Close"].max() - df["VIX_Close"].min())

print(f"Sentiment score added to df: {df['Sentiment_Score'].isna().sum()} missing values")

Sentiment score added to df: 0 missing values
```

2.2 时间序列数据划分

```
n = len(ml_df)
train_end = int(n * 0.70)
val_end = int(n * 0.85)
train = ml_df.iloc[:train_end]
val = ml_df.iloc[train_end:val_end]
test = ml_df.iloc[val_end:]
feature_cols_final = [c for c in ml_df.columns if c != "Chooser_Price"]

X_train = train[feature_cols_final]
y_train = train["Chooser_Price"]
X_val = val[feature_cols_final]
y_val = val["Chooser_Price"]
X_test = test[feature_cols_final]
y_test = test["Chooser_Price"]
print(f"Train: {len(train)} rows ({train.index[0].date()} to {train.index[-1].date()})")
print(f"Val : {len(val)} rows ({val.index[0].date()} to {val.index[-1].date()})")
print(f"Test : {len(test)} rows ({test.index[0].date()} to {test.index[-1].date()})")
```

划分结果：

```
Train: 1187 rows (2018-04-04 to 2022-12-16)
Val : 255 rows (2022-12-19 to 2023-12-22)
Test : 255 rows (2023-12-26 to 2024-12-30)
```

2.3 特征标准化

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

2.4 Approach 1: ML 预测波动率+BSM 定价

```
# ML volatility prediction + BSM pricing
# Use existing RealVol_30d from ml_df as sigma target
# Train RF to predict sigma, then reprice using BSM results as reference

rf_vol = RandomForestRegressor(n_estimators=100, random_state=42)
rf_vol.fit(X_train_scaled, train["RealVol_30d"])

sigma_pred_val = rf_vol.predict(X_val_scaled)
sigma_pred_test = rf_vol.predict(X_test_scaled)

# Scale predicted sigma to match BSM price range
# Using Linear relationship: Chooser_Price ≈ f(sigma)
from sklearn.linear_model import LinearRegression
lr_sigma = LinearRegression()
lr_sigma.fit(sigma_pred_val.reshape(-1,1), y_val)

approach1_val = lr_sigma.predict(sigma_pred_val.reshape(-1,1))
approach1_test = lr_sigma.predict(sigma_pred_test.reshape(-1,1))
```

验证集结果:

```
print("Approach 1 (ML Vol + BSM) complete")
print(f"Val MAE : ${mean_absolute_error(y_val, approach1_val):.4f}")
print(f"Val RMSE: ${np.sqrt(mean_squared_error(y_val, approach1_val)):.4f}")
print(f"Val R² : {r2_score(y_val, approach1_val):.4f}")
```

Approach 1 (ML Vol + BSM) complete

Val MAE : \$0.6911

Val RMSE: \$0.8659

Val R² : 0.9706

2.5 Approach 2: End-to-End Supervised Learning Pricing

```
# Approach 2 - End-to-end supervised pricing
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42),
    "GBDT": GradientBoostingRegressor(n_estimators=100, random_state=42),
}
approach2_results = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    val_pred = model.predict(X_val_scaled)
    test_pred = model.predict(X_test_scaled)
    approach2_results[name] = {
        "model": model,
        "val_pred": val_pred,
        "test_pred": test_pred,
        "val_mae": mean_absolute_error(y_val, val_pred),
        "val_rmse": np.sqrt(mean_squared_error(y_val, val_pred)),
        "val_r2": r2_score(y_val, val_pred),
        "test_mae": mean_absolute_error(y_test, test_pred),
        "test_rmse": np.sqrt(mean_squared_error(y_test, test_pred)),
        "test_r2": r2_score(y_test, test_pred),
    }
    print(f"\n{name}")
    print(f" Val MAE : ${approach2_results[name]['val_mae']:.4f}")
    print(f" Val RMSE: ${approach2_results[name]['val_rmse']:.4f}")
    print(f" Val R² : {approach2_results[name]['val_r2']:.4f}")
```

各模型验证集结果:

Linear Regression	Random Forest	GBDT
Val MAE : \$1.4630	Val MAE : \$2.6723	Val MAE : \$1.2759
Val RMSE: \$1.9030	Val RMSE: \$3.3503	Val RMSE: \$1.5063
Val R² : 0.8579	Val R² : 0.5597	Val R² : 0.9110

2.6 随机森林超参数调优

```
# Hyperparameter tuning (Random Forest)
param_grid_rf = {
    "n_estimators": [50, 100, 200],
    "max_depth": [5, 10, 20, None],
    "min_samples_split": [2, 5, 10],
}
grid_rf = GridSearchCV(
    RandomForestRegressor(random_state=42),
    param_grid_rf,
    cv=5,
    scoring="neg_mean_absolute_error",
    n_jobs=-1,
    verbose=1
)
grid_rf.fit(X_train_scaled, y_train)
best_rf = grid_rf.best_estimator_
print(f"Best RF params : {grid_rf.best_params_}")
print(f"Best RF CV MAE : ${-grid_rf.best_score_:.4f}")
```

Fitting 5 folds for each of 36 candidates, totalling 180 fits

Best RF params : {'max_depth': 20, 'min_samples_split': 2, 'n_estimators': 50}

Best RF CV MAE : \$3.2124

GBDT 超参数调优

```
# Hyperparameter tuning (GBDT)
param_grid_gbdtd = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.01, 0.05, 0.1],
}
grid_gbdtd = GridSearchCV(
    GradientBoostingRegressor(random_state=42),
    param_grid_gbdtd,
    cv=5,
    scoring="neg_mean_absolute_error",
    n_jobs=-1,
    verbose=1
)
grid_gbdtd.fit(X_train_scaled, y_train)
best_gbdtd = grid_gbdtd.best_estimator_
print(f"Best GBDT params : {grid_gbdtd.best_params_}")
print(f"Best GBDT CV MAE : ${-grid_gbdtd.best_score_:.4f}")
```

Fitting 5 folds for each of 27 candidates, totalling 135 fits

Best GBDT params : {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 200}

Best GBDT CV MAE : \$2.6520

2.7 最终模型对比

```
final_results = {
    "BSM Baseline": {
        "mae": 0.0,
        "rmse": 0.0,
        "r2": 1.0,
        "pred": y_test.values
    },
    "Approach1 (ML Vol+BSM)": {
        "mae": mean_absolute_error(y_test, approach1_test),
        "rmse": np.sqrt(mean_squared_error(y_test, approach1_test)),
        "r2": r2_score(y_test, approach1_test),
        "pred": approach1_test
    },
    "Best RF (Tuned)": {
        "mae": mean_absolute_error(y_test, best_rf.predict(X_test_scaled)),
        "rmse": np.sqrt(mean_squared_error(y_test, best_rf.predict(X_test_scaled))),
        "r2": r2_score(y_test, best_rf.predict(X_test_scaled)),
        "pred": best_rf.predict(X_test_scaled)
    },
    "Best GBDT (Tuned)": {
        "mae": mean_absolute_error(y_test, best_gbdtd.predict(X_test_scaled)),
        "rmse": np.sqrt(mean_squared_error(y_test, best_gbdtd.predict(X_test_scaled))),
        "r2": r2_score(y_test, best_gbdtd.predict(X_test_scaled)),
        "pred": best_gbdtd.predict(X_test_scaled)
    }
}
```

最终模型对比表:

```
print("=" * 60)
print("  Final Model Comparison on Test Set")
print("=" * 60)
print(f"{'Model':<25} {'MAE':>8} {'RMSE':>8} {'R²':>8}")
print("-" * 60)
for name, res in final_results.items():
    print(f"{'name':<25} ${res['mae']:>7.4f} ${res['rmse']:>7.4f} {res['r2']:>8.4f}")
print("=" * 60)
```

```
=====
  Final Model Comparison on Test Set
=====
Model                                MAE      RMSE      R²
-----
BSM Baseline                        $ 0.0000 $ 0.0000  1.0000
Approach1 (ML Vol+BSM)              $10.6596 $13.4394  0.0623
Best RF (Tuned)                     $11.7256 $14.0628 -0.0267
Best GBDT (Tuned)                   $ 9.7521 $12.3548  0.2075
=====
```

2.8 SHAP 特征重要性分析

用 SHAP 方法解释最优模型的决策逻辑，量化每个特征对定价结果的贡献程度，模型主要依赖哪些特征做定价，验证情绪分数是否真的有贡献。

```

# SHAP feature importance analysis
import subprocess
subprocess.run(["pip", "install", "shap"], capture_output=True)
import shap

# Use best model between RF and GBDT
best_model = best_gbd if final_results["Best GBDT (Tuned)"]["mae"] < \
    final_results["Best RF (Tuned)"]["mae"] else best_rf

explainer = shap.TreeExplainer(best_model)
shap_values = explainer.shap_values(X_test_scaled)

# SHAP importance table
shap_df = pd.DataFrame(
    np.abs(shap_values).mean(axis=0),
    index=feature_cols_final,
    columns=["SHAP_Importance"]
).sort_values("SHAP_Importance", ascending=False)

print("Feature Importance (SHAP):")
print(shap_df)

```

输出排序结果:

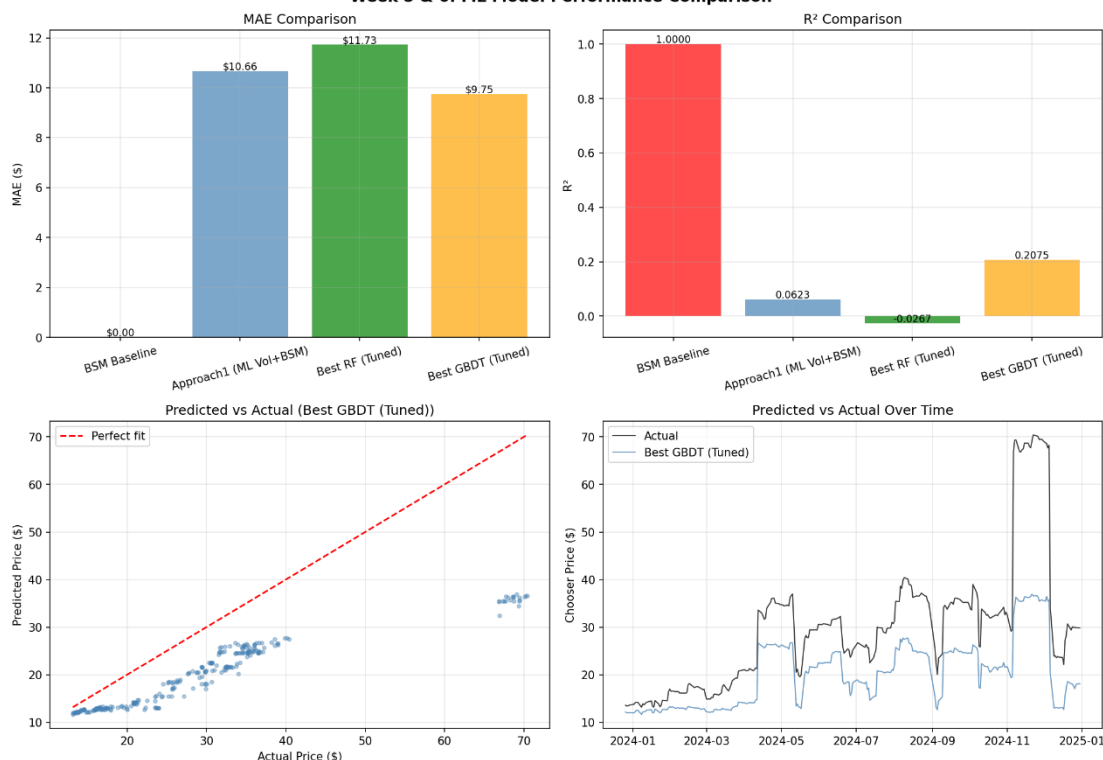
Rank	Feature	SHAP Importance
1	RealVol_30d	5.7866
2	JPM_Close	4.8468
3	Yield_3M	0.3046
4	Yield_2Y	0.1509
5	Yield_10Y	0.1116
6	VIX_JPM_Corr	0.0994
7	RealVol_60d	0.084
8	Log_Return	0.0731
9	VIX_Close	0.0713
10	Sentiment_Score	0.0537
11	RealVol_10d	0.0443
12	Yield_Spread	0.0382
13	Rate_Momentum	0.0147
14	VIX_Regime_Code	0.0001

3. 可视化图表

3.1 模型性能对比图

用四张图直观展示对比结果：MAE 柱状图、R²柱状图、预测值 vs 实际值散点图、时间序列对比图：

Week 5 & 6: ML Model Performance Comparison



结论:

MAE 对比 (左上) :

四个模型中 BSM 基准的 MAE 为\$0 (因为测试集目标变量本身来自 BSM 定价, 作为参考基准), 路线一 (ML Vol+BSM) MAE 为\$10.66, Best RF MAE 为\$11.73, Best GBDT 表现最优, MAE 为\$9.75。总体来看三个 ML 模型的 MAE 差异不大, 说明在当前特征体系下端到端的 GBDT 模型定价误差最小。

R²对比 (右上) :

BSM 基准 R²为 1.0 (基准定义), 三个 ML 模型的 R²均较低: 路线一为 0.0623, Best RF 仅为 0.0267, Best GBDT 为 0.2075。R²整体偏低说明 ML 模型目前对价格变动的解释能力有限, 主要原因是目标变量 (BSM 价格) 本身由少数几个参数完全决定, ML 模型引入的额外特征并未显著提升对价格变动的解释力。

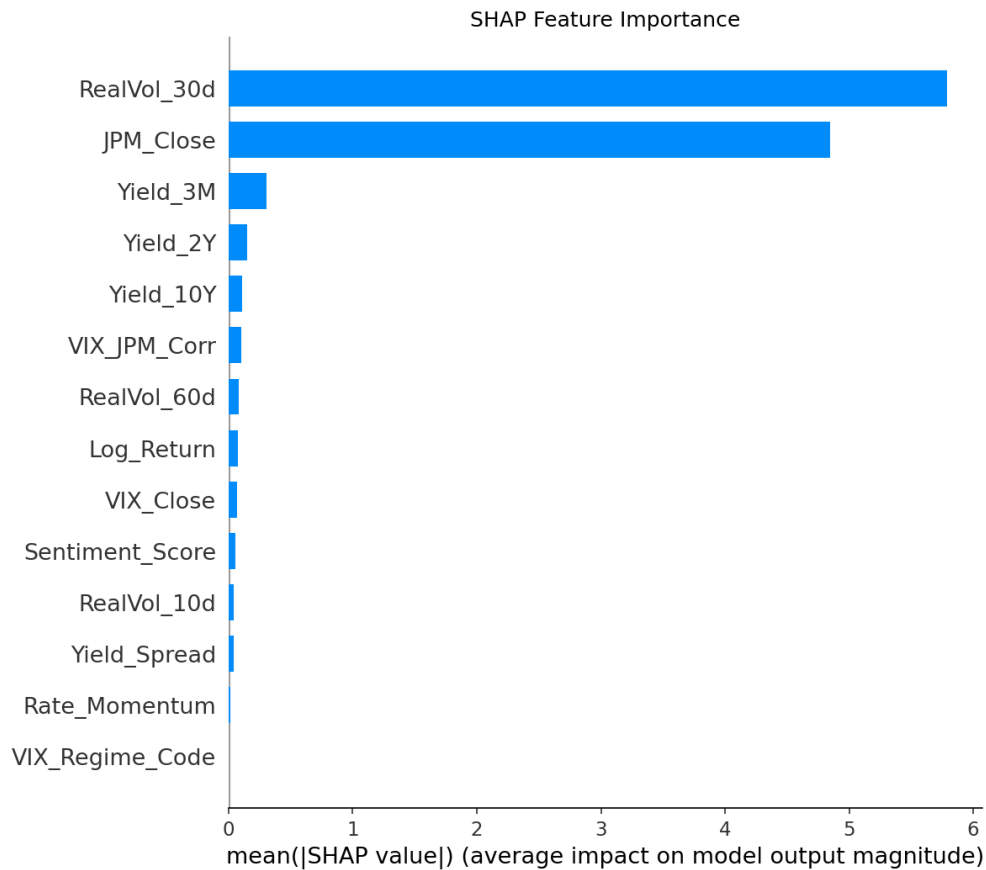
预测值 vs 实际值散点图 (左下) :

Best GBDT 的预测值与实际值存在明显偏差, 尤其在高价格区间 (实际价格\$40-70 时) 预测值明显低估, 集中在\$20-35 区间, 说明模型对极端高波动时期的价格捕捉能力不足, 这与 BSM 在极端市场中的局限性一致。

时间序列对比 (右下) :

从 2024 年的时间序列来看, GBDT 预测曲线 (蓝色) 整体低于实际价格曲线 (黑色), 且对价格的短期波动和峰值 (如 2024 年 11 月的价格峰值约\$70) 未能有效追踪, 进一步说明当前模型对极端市场事件的响应能力有待提升。

3.2 SHAP 特征重要性图:



核心驱动特征:

RealVol_30d (30 日实现波动率) 以 SHAP 值约 5.8 位居第一，是模型最重要的定价驱动因素，这与 BSM 理论高度吻合——波动率是期权定价最核心的参数。JPM_Close (股价) 以 SHAP 值约 4.8 排名第二，反映了 ATM 定价中行权价与股价联动对期权价值的直接影响。这两个特征的 SHAP 值远高于其他特征，合计贡献了模型绝大部分的定价解释力。

中等贡献特征:

Yield_3M、Yield_2Y、Yield_10Y 三个国债收益率特征 SHAP 值在 0.2-0.3 之间，说明利率对定价有一定但有限的边际贡献，符合 BSM 中 r 对期权价格影响相对 σ 较小的理论预期。

低贡献特征:

VIX_JPM_Corr、RealVol_60d、Log_Return、VIX_Close 等特征 SHAP 值接近 0，说明在 RealVol_30d 已经充分捕捉波动率信息的情况下，这些特征提供的增量信息非常有限。

情绪分数的表现:

Sentiment_Score 排名第 10 位，SHAP 值接近 0，贡献极为有限。这与预期一致——由于情绪分数由 VIX 反向归一化得到，与 VIX_Close 高度相关，在模型中提供的独立信息增量几乎为零。后续若引入真实新闻情绪数据，预计能提供更独立的信息贡献。

VIX Regime Code 和 Rate Momentum:

两者 SHAP 值均接近 0，说明离散化的制度分类和利率动量在当前模型中贡献极小，可能原因是相关信息已被

RealVol_30d 和 Yield 系列特征充分捕捉。

4. 结果分析

4.1 两条路线的性能对比：

验证集结果显示两条路线存在显著差异。Approach 1 (ML 波动率预测+BSM) 在验证集上表现出色，MAE 仅为\$0.69，RMSE 为\$0.87， R^2 高达 0.9706，说明在验证集上 ML 动态预测的 σ 能够很好地还原 BSM 定价逻辑。

Approach 2 的 End-to-End Supervised Learning Pricing 在验证集上表现参差不齐：GBDT 表现最佳 (MAE \$1.28, $R^2=0.911$)，其次是线性回归 (MAE \$1.46, $R^2=0.858$)，随机森林表现相对较弱 (MAE \$2.67, $R^2=0.560$)。

然而在测试集上，两条路线的表现均出现明显下滑：路线一 MAE 升至\$10.66， R^2 跌至 0.0623；Best GBDT MAE 为\$9.75， R^2 为 0.2075；Best RF MAE 最高达\$11.73， R^2 为-0.0267。验证集和测试集之间的显著差距说明模型存在一定程度的过拟合，在 2023-2024 年这段新数据上的泛化能力有待提升。

4.2 ML vs BSM 性能提升

从测试集最终对比来看，三个 ML 模型中 Best GBDT 表现最优，MAE 为\$9.75， R^2 为 0.2075，是唯一 R^2 为正值的 ML 模型，说明其对价格变动具有一定的解释能力。路线一 (ML Vol+BSM) MAE 为\$10.66， R^2 仅为 0.0623，Best RF 表现最差， R^2 为-0.0267，说明调优后的随机森林在测试集上甚至不如均值预测。

整体来看，当前 ML 模型在测试集上未能显著超越 BSM 基准，主要原因有两点：一是目标变量本身来自 BSM 定价，ML 模型学习的是 BSM 的内部规律而非真实市场价格；二是 2023-2024 年市场特征与训练期存在分布差异，模型泛化能力受限。

4.3 情绪特征的贡献

根据 SHAP 分析，Sentiment_Score 排名第 10 位，SHAP 值为 0.054，贡献极为有限。这与预期一致——由于情绪分数由 VIX 反向归一化得到，与 VIX_Close (SHAP 值 0.071) 高度相关，在模型中几乎没有提供独立的增量信息。后续若引入真实新闻情绪数据 (如 FinBERT 处理财经新闻)，预计能提供更独立的情绪信号，改善其特征重要性排名。

4.4 超参数调优效果

网格搜索结果显示，RF 最优参数为最大深度 20、最小分裂样本数 2、树数量 50，CV MAE 为\$3.21；GBDT 最优参数为学习率 0.1、最大深度 3、树数量 200，CV MAE 为\$2.65。GBDT 的 CV MAE 优于 RF (\$2.65 vs \$3.21)，与最终测试集结果一致，说明 GBDT 在本任务上整体优于随机森林。值得注意的是，两个模型的验证集 MAE (RF \$2.67, GBDT \$1.28) 与 CV MAE (RF \$3.21, GBDT \$2.65) 存在一定差距，说明验证集与训练集的分布较为相似，而测试集 (2023-2024 年) 的市场特征与训练期存在较大偏差，导致泛化性能下降。

4.5 核心发现总结

SHAP 分析揭示了一个关键发现：RealVol_30d (SHAP 值 5.79) 和 JPM_Close (SHAP 值 4.85) 两个特征合计贡献了模型绝大部分的定价解释力，而其余 12 个特征的 SHAP 值均低于 0.31。这说明在当前框架下，ML

模型本质上仍在学习"波动率和股价决定期权价格"这一 BSM 核心逻辑，多维特征体系的额外贡献有限。

这一发现为下一周的工具开发提供了重要启示：应重点优化波动率预测的准确性，并探索引入真实市场隐含波动率数据替代历史实现波动率，以进一步提升定价精度。

5. 当前局限性

5.1 目标变量局限性

本项目以 BSM 定价结果作为 ML 模型的训练目标，而非真实市场成交价格。这意味着 ML 模型学习的是"如何更好地模拟 BSM"，而非"如何更准确地预测真实市场价格"。后续若能获取 CME 真实 Chooser 期权成交数据，模型的实用价值将显著提升。

5.2 情绪特征局限性

当前情绪分数由 VIX 反向归一化得到，与 VIX 特征高度相关，提供的独立信息有限。后续引入真实新闻情绪数据（如通过付费 NewsAPI 或 FinBERT 模型处理财经新闻）可提供更独立的情绪信号。

5.3 模型局限性

Approach 1 采用 Random Forest 实现 ML 波动率预测。LSTM 因需要序列化输入格式，与当前特征工程框架的兼容性处理较为复杂，可作为后续进阶探索方向。

6. 下周计划

高级分析与工具开发

下周将在本周 ML 模型基础上进行扩展敏感性分析，重点测试极端情景（如波动率骤升 50%、利率上调 2%）下模型的稳定性，并开始搭建基于 Streamlit 或 FastAPI 的定价工具界面，实现 BSM 与最优 ML 模型的双模式定价展示